

Check Point Task Notes

Food ordering

- An enhanced switch must:
 - **Return** a value if it's assigned to a variable
 - A switch expression must return/yield a value if assigned to a variable, but you can still print inside block cases before yielding."
- When using a **retry, while (true) {** loop, you can use multiple it to assign values to multiple variables. For example, if you made a method to get a name and a title via scanner, you could declare String name and String title outside the retry loop and then assign them both within the loop. You DON'T need separate retry loops for each variable. Multiple variables can be assigned within the same loop.
- The **break** keyword is used to EXIT a loop or switch statement prematurely, i.e., before the whole thing has run, usually upon successful input you would break out of a retry loop. It terminates the loop or switch and transfers control to the statement following loop or switch
- The **continue** keyword is used to SKIP the current iteration of the loop and proceed to the next iteration. Usually used in guards for invalid user input, it will skip the rest of the loop and start the next iteration, effectively taking you back to the start upon invalid input which allows the user to try again without exiting the program. It doesn't end or break out of the loop though, since the condition to end has not been met, it just takes you back to the start
- You usually need to use **scanner.nextLine()** after reading numerical input like **x = scanner.nextInt()** or **scanner.nextDouble()** if the next input will use **nextLine()**, because **nextInt()** leaves the newline character in the buffer. This is because **nextInt()** leaves the enter key (for the user entering the input) in the input buffer and if you don't clear it, the next user input section will read an empty String. So, **scanner.nextLine** will create an empty new line and prevent input bugs, so we USUALLY use this after reading numerical input. An example of when you don't need it is if **scanner.nextInt()** is followed by **scanner.nextInt()**. **So always use if reading numerical input is followed by reading String input**
- When using a menu or trying to accept user input to make a choice, i.e., "type \n1 to choose x or \n2 to choose y", you need to follow this up with an IF statement or switch to decide what happens next
- Loops control program flow, not just retrying or repetition and can be used in main with methods to control the program
- For switch expressions vs IF/ELSE statements:
 - **Switch is better when:**
 - You have a fixed set of numbered choices, like a menu
 - **IF/ELSE is better when:**
 - If the logic is based on conditions like number ranges, comparisons, yes/no decisions or validation requiring complex rules
 - For example, if price > 50 = discount or if name == empty then retry.
- You can't return or yield multiple variable types, like a String and a double from the same switch expression if you've declared the variable type you want to

return before creating the switch, for example double starterChoice = switch starter. A switch must return one single type, in this case a double

- To work around this if you do want to return 2 different variable types, declare them outside the switch and assign them inside it i.e.,

```
○ String starterName;  
○ double starterPrice;  
○ switch (starter) {  
○ case 1 -> {  
○ starterName = "Garlic bread";  
○ starterPrice = 4.50;
```

- To add variables to an arrayList, use .add. following on from the previous example
 - Orderitems.add(starterName);
 - NAMEOFARRAYLIST.add(VARIABLENAME)
- You often don't need to use the default keyword in a switch if all possible values are already covered through earlier validation. It is only required if you want to handle unexpected values, or invalid user input. If all possible values are already covered via previous input validation, you can omit this.

Orders/food menu flow

- Before asking for any food choices, I've asked the user how many orders they would like to place in a method which returns an int between 1 and 5. This is just a simple while (true) loop allowing for retries if the guard conditions are breached, but this is important for the program as a whole. This int is stored in a variable called **orders**.
- Again, before the starters are selected, **this whole section is wrapped in an outer FOR loop**. This is just a for loop at the start and is before, so outside the full section, **allowing for the starter choice segment to loop until the condition is met to exit the loop**, this condition being when it has looped the same amount of times as the number stored in **orders**, i.e., **when times looped = orders**. This looks like:
 - **for (int orderNumber = 0; orderNumber < orders; orderNumber++) {**
 - orderNumber starts at 0 as loop iterations start at 0, (so 5 loops is loop 0,1,2,3,4) and orderNumber++ means **keep looping until the condition is met, the condition being loop while order number IS LESS THAN orders** (the int which the user entered for how many orders they want to place).
 - **This is the furthest out loop, or outermost loop, so this is ended last as we want this whole section to keep running until the order condition has been met, which is why it comes first.**
- **Then** we have Boolean starterConfirmed = false;. This is known as a **flag** and is used as a **condition to break out of a loop**. In general terms if you wanted a retry loop in a method like asking the user if they want to return again, your method will return a Boolean. Like true, I am happy and don't want to run the program again or false, I do want to run the program again. These are interchangeable based on if you use ! next to the flag, i.e., **while(!exitProgram)** which neabs keep looping if while the boolean is not true, or false, and will exit if the boolean is assigned to true since the loop condition is no longer met. . This just depends on the wording of your program and what you set the initial Boolean flag = to, either true or false, and one can be more useful than the other in certain scenarios but it's mainly down to how you word them. Typically, boolean variable flags and while loops allow reruns. In this case it is an outer loop, outside the block of code that you want to allow a rerun in, the whole section which starts before the allergy section runs and at the end you will give them the choice to choose again or proceed, which is where the loop (} – END OF WHILE LOOP) will end. Usually, you start by declaring a Boolean variable = to false and below this, while (variable name) {, which just means while this value is true, keep looping. At the end of the block you want to run, you would have if (variable name), and then below this the break; keyword, which means if this value is true, exit the loop and proceed with the rest of the program. **In this case, we didn't need the while (variable name) because the rerun was confirmed by the IF statement at the end.**
 - **In** this case it's used to allow the user to rerun the whole ordering section for each course which is why the Boolean is declared here and as false, as if the user isn't happy with their starter, it will remain as false as seen in the IF statement near the end of this section. The next IF statement before the outer loop ends will **ONLY BREAK THIS OUTER LOOP IF THE VALUE IS NOW TRUE,**

which is why if its false, it will rerun. This is placed at the start after the for loop and ended at the end only before the for loop because this allows the user to rerun this entire section if they want to, so it must be placed before the section begins to achieve this flow.

- In this example, it is used as there was an issue with the user retrying based on any order after the first, so if they'd placed 3 orders and weren't happy with their second, it would stick them in an infinite loop so this was used to break out of the loop.
- **After** this it's just a **while (true)** rerun loop acting as an inner loop again to allow the allergy section to validate user input and reattempt if it failed.
- **Next** again is just a regular **while loop** to allow a rerun of the starterConfirmation section for valid input but this also assigns a value to this variable to allow a rerun of the whole section based on if the user is happy to confirm their order. This one is unique though as its just a flow control method, and this loop is broken out of either way, but the variable being assigned to allow a rerun is confirmed in the last IF statement.
- **Then** if they are happy to proceed, we end the outer loop surrounding the whole thing in the final IF statement based on the value assigned in the previous while loop.
- **Then** we end the for loop after it has looped as many times as the orders variable

Orders/food menu flow code

```
○ for (int orderNumber = 0; orderNumber < orders; orderNumber++) {  
  //FOR LOOP to allow multiple reruns until rerun amount = orders  
  //placed  
  Boolean startedConfirmed = false;  
  
  While true { //OUTER LOOP  
  
    While (true) { //inner loop  
      Try { //INNER to allow us to catch invalid input  
        //allergy options  
        break; //breaks out of INNER loop  
      } catch  
        //catch statement  
      } end catch  
    } end INNER  
  
    //switch statement  
    //Block of code  
    //end switch  
  
    Declare starterConfirmation;  
    While (true) { //another INNER loop for this block of code  
      Try {  
        //starter confirmed either TRUE or FALSE but  
        //BREAK out of inner loop upon either choice  
        //as it BREAKS out of THIS INNER LOOP ONLY  
        //and the value is dealt with in the below IF  
      } catch  
        //catch statement  
      } end catch  
  
      } end INNER  
  
      IF (starterConfirmed) { //IF starterConfirmed is TRUE  
        //add to arrayList  
        break; //BREAK out of OUTER LOOP  
      } //END IF  
    } //END OUTER LOOP  
  } //END FOR LOOP
```

- Due to this being an earlier project the code isn't very clean and most of it is in main, because of this I haven't used methods for each section, so this has resulted in using nested loops to achieve the result. Although it works, it's quite unclear and hard to read, as well as debug. Remember, most of your code should be in methods and main is used to control the flow of the program. Here, the for loop and outer while loop could be in main and then everything else as separate methods, as these 2 loops control the flow, loop x number of types and if user is unhappy, retry. Everything is just code to actually execute so these could be helper methods.
- The for loop section could also have been done with another while (true loop) by adding it in the same position as the for the loop and at the end asking, would you like to order another starter, please select 1 or 2 and an IF/ELSE statement with if ==1, break; to break out of the loop as they wouldn't like another starter and else ==2 continue; if they would like another start, with the continue statement taking you all the way back to the start since this is the loop that comes first. However, the for loop is a much cleaner method. The only difference is that we would add ArrayList?????????
- The break; keyword will exit a loop, typically used when a correct input has been received or a condition has been met, like here, the user is happy with their menu choice so break out of the loop as the rest doesn't need to run and proceed
- The continue; keyword is used to go back to the start of the loop, or more accurately, skip the remainder of the code in the loop and start the next iteration of the loop, effectively taking you back to the start. This is used when conditions have not been met and you want to take the user back to the start of loop to try again. For example, please choose \n1 or \n2 and the user types 3, although it is a valid input, an int, it's out with the accepted input set in the guard clause so disregard this and let them try again.
- The loop with starterConfirmation is unique because this is all nested, this loop is within an outer while loop which is inside a for loop. So, in isolation on a retry loop, you would have continue to let the user retry but here, if you used continue; on the user returning a false value, i.e., they don't want to proceed and want to retry, you would only restart THIS specific loop, so they would be stuck in this loop asking them to confirm if they want to proceed or not. This is why we have used a break; for each answer. Upon either input the user will break out of this loop only, but the value will be changed for the initial flag variable, starterConfirmed, either true or false.
- The following IF statement controls the flow based on the value the previous loop returned so if(starterConfirmed), i.e., if this value is true, NOW break out of the outer loop and proceed to the next section. However if this value is false. i.e., the user chose 2 and wanted to reselect, the previous inner while loop will still be broken out of because of the break;; however the value will not be updated for the starterConfirmed flag variable and it will remain as false, and since the only condition to break the whole outer loop is if starterConfirmed is true, then this loop will run again anyway. There is no need for continue; here as this is at the end of loop, so if the condition to break out is not met, it will run again anyway.

- In an IF/ELSE statement like the one here with choose 1 to proceed or 2 to retry, you don't need to use a guard for conditions like `if x > 2 && x < 1` to ensure the input is valid because this is already covered in the ELSE statement. This is because the loop here is saying return a value IF `x = 1` break and return a different value if `x = 2` break, so there is nothing else the answer can be due to this loop **ONLY BEING EXITED IF 1 or 2 are returned as these are the only conditions to break out of the loop, the loop will restart anyway, but you should print a message after the else like please choose 1 or 2. You would still need a try/catch to handle any invalid input.**
- Java error messages will usually cover things like "this condition is always true or false" or "unreachable branch", meaning the code won't execute as it never gets to that stage of the code due to something else.
- You can delete a class on IntelliJ by right clicking it and selecting delete
- The Boolean `starterConfirmed` must be inside the for the loop because if there are multiple orders, it must be reset each time the loop runs. If it was outside, then its value would change after the 1st iteration of the for loop has run and its value has been set to true, which would never change and wouldn't let the user retry on any order but their 1st since it was outside meaning it was unreachable. This is a problem I encountered as originally, I had it outside, which meant if the user wanted to place multiple orders, but then weren't happy with their choice on the 2nd 3rd or 4th etc., and wanted to reselect, it wouldn't let them and would continue on anyway. This is because to exit the outer while loop, `starterConfirmed` must be set to true at the end, but since it was outside the for loop, after it was set to true after the 1st order, it IS NEVER RESET and remains true throughout the 2nd, 3rd 4th etc., so the retry wouldn't work and it would proceed anyway. Basically the for loop runs as many times to achieve the value of orders placed, so this will run x amount of times anyway, and the outer while loop runs until it is broken out of when the `starterConfirmed` variable is true, but it's the for loop which is the main control loop for the whole thing since it's the outermost loop or first loop written, so since `starterConfirmed = false` was OUTSIDE this, after it is updated to true, it is never reset back to false after the 1st iteration of the loop. So, on any subsequent iterations, the flag is always true and since this is the case, it will always break out of the outer while loop at end since the condition to break out of it is always met. This is because the the first line of code to run after the first iteration of multiple orders was the starter selection part again, since the flag was outside the loop and now became unreachable/inaccessible due the program still being inside the for loop and the flag being outside, so it couldn't be reset to allow for retries, since it was set as true after the first loop and after that, it became unreachable as it was outside the for loop. To fix this, it was moved inside the for loop so every time the outer while completes and is broken out of, and resets back to the first line of code in the loop, the first thing that ran now is `starterConfirmed = false`, resetting the flag so that the outer while loop condition has to be met again, then the menu display, meaning on every iteration of the loop, the value of the flag was reset back to false so it was not automatically met and if the user wanted to retry which wouldn't change the value in the first place, it would actually work now because the flag was still false.

Allergy exclusion system

- To set this up I used an int allergy and asked if the user had an allergy and then saved the answer in the int variable, 1 being no allergies 1 and then 2-7 being different allergies.
- I then declared a Boolean for each allergy, 2-6 and set them as false as the baseline would be they DID NOT have an allergy and if they did, then the corresponding value would be changed to true.
- Then I used a switch, `switch (allergy) {`, and used each case to update the values. The first case didn't change anything as this was no allergies, and since the allergies were set to false anyway the user wouldn't be prevented from ordering anything. Then `case 2 -> {` would set the garlicAllergy Boolean (declared before the switch) = to true which would later prevent them ordering anything with garlic. The next cases followed the same pattern until the last case, 7, which just printed a message saying we couldn't accommodate anything else, basically a disclaimer like a real website, but this in effect was the same as the 1st case.
- Then, when I displayed the starter choices and asked the user to choose a starter from the 3 set options, I used an if statement with ! to actually display the choices, `if(!garlicAllergy) {`, print 1 for garlic bread. I did this because it meant if the user DID not have a garlic allergy, or garlicAllergy = false;, ONLY THEN display this option. If garlicAllergy was true, then this option would NOT be displayed because they should not be able to choose this if they did have an allergy. Then I did the same for the other 3 starter allergies.
- Next I set variable int starter and set it equal to the user input (this was already initialised outside the loop so it was accessible after), either 1, 2 or 3 for the set starters (I'll cover the custom stuff later)
- Then I used another set of if statements to actually prevent the user from choosing something they were allergic to. I previously hid options the user was allergic to from printing, however if they were allergic to garlic bread, although they option was hidden, they could still choose 1 to select it. To prevent this from happening I set up the following if statement:

```
If (starter == 1 && garlicAllergy) {  
    Print "you're allergic so can't choose this" //Not exactly this but...  
    continue;  
}
```

What this done was say if the user chooses option 1 for garlic bread AND ALSO garlic allergy = true; (they are allergic), print a message saying they cant choose this and the continue statement will skip the rest of the loop and take them back to reselect another starter

If none of these if statements were the case, i.e., valid input, it would break out of the loop and save the starter choice.
- Below is a snippet of this code

```

scanner.nextLine();
//Again, declare variables outside the switch statement as you can't declare them inside and assume no allergies
boolean garlicAllergy = false;
boolean lentilAllergy = false;
boolean seafoodAllergy = false;
boolean peanutAllergy = false;
boolean dairyAllergy = false;

switch (allergy) { //Switch to either allow user to order anything from the menu or block certain menu choices if they are allergic to it
    case 1 -> {
        System.out.println("Perfect, you can order anything from the menu!"); //Allow user to choose any item
    }
    case 2 -> {
        //If they are allergic, update boolean value and inform the user their dietary requirements have been noted
        garlicAllergy = true;
        System.out.println("Garlic allergy recorded.");
    }
    case 3 -> {
        lentilAllergy = true;
        System.out.println("Lentil allergy recorded.");
    }
    case 4 -> {
        seafoodAllergy = true;
        System.out.println("Seafood allergy recorded.");
    }
}

```

First section using the switch to return true to the corresponding allergy if the user had allergies

```

while (true) { //Inner retry loop
    try {
        System.out.println("For your starter, please choose from the following:");
        if (!garlicAllergy) { // If user has garlic allergy, DO NOT display garlic bread as an option
            System.out.println("1 For Garlic Bread");
        }
        if (!lentilAllergy) {
            System.out.println("2 For Lentil Soup");
        }
        if (!seafoodAllergy) {
            System.out.println("3 For Prawn Cocktail");
        }
        System.out.println("4 For Custom Dish (These are more expensive)"); //Allow user to choose custom dish
        System.out.println("5 To skip starter"); //Allow user to skip course
        starter = scanner.nextInt();

        if (starter == 1 && garlicAllergy) { //If user is allergic, do not let them order the item
            System.out.println("Sorry, but you're allergic to this item so you can't order it.");
            continue;
        }
        if (starter == 2 && lentilAllergy) {
            System.out.println("Sorry, but you're allergic to this item so you can't order it.");
            continue;
        }
        if (starter == 3 && seafoodAllergy) {
            System.out.println("Sorry, but you're allergic to this item so you can't order it.");
            continue;
        }
        if (starter < 1 || starter > 5) { //Guard to ensure input is within range
            System.out.println("Please choose one of the allocated options.");
            continue;
        }
        break;
    } catch (Exception e) { //Catch invalid input
        System.out.println("Please enter a whole number from 1 to 5.");
        scanner.next();
    }
}

```

If statements both hiding the menu item and preventing the user from choosing an item if they were allergic to it, i.e., if any if the Boolean values were not true

Food selection switch

- The food selection switch works by setting the user choice as a variable, int starter in a previous int statement.
- It is then a switch expression `switch (starter) {` with cases assigning different values to the variable starterName (String) and starterPrice (double), which were also assigned earlier.
- For example, case 1 which is garlic bread, would assign the value garlic bread to String starterName and the value 4.50 to double starterPrice, as below:

```
Switch (starter) {  
Case 1 -> {  
    starterName = "garlic bread";  
    starterPrice = 4.50;  
} //End case 1
```

- Case 4 which is the only different one is very similar but just adds a bit extra.

```
Case 4 -> {  
    Print "Enter custom starter name"; //Lasagne for example  
    starterName = scanner.nextLine();  
    starterPrice = 11.50; //just a set price for custom orders  
} //End case 4
```

- Case 5 which is for no starter works exactly the same as the rest of the cases, but just assigns the name "no starter" to starterName and 0 to starterPrice, thus not adding anything to the total bill at the end
- You can't declare a variable inside a switch statement and yield it as a value, you must declare the variable outside the switch statement, i.e., Boolean dairyAllergy = false; and then reassign its new value true, inside the switch, similar to a while loop where you initialise a variable outside. This also means the variable is accessible everywhere, and not just inside the switch, which would be another problem.

Method best practice

- It doesn't matter in Java if the method is written above or below the main file, but it's good to write them below at the end in the style I'm used to.

- For example:

```
Public class foodOrdering {  
    Public static void main(String[] args) {  
        //main  
        //main should only be calling your methods and flow control stuff  
        //like using an IF statement to decide which method to call based  
        //on what the user has chosen like in my banking project  
    } end main  
  
    /**  
    ** Helper methods defined below  
    **/  
  
    Private static void choose starter {  
    }  
  
    Private static int.... {  
    }  
  
    //and so on.....  
  
} END
```

- Try to keep main like the index of a story and the methods are your chapters, main should be short and sweet with flow control and calling methods and all the meaty code is hidden away in your methods
- You really only want your method to do **1 job** as remember, they should only be returning values, printing things, updating values etc. Methods are usually quite short and although they can be long, it is important not to overcomplicate them. For example, your method might require multiple loops to get the value you want to return like a for and while or 2 whiles, like first making an int to convert to a boolean or like the starter selection section for example where its long and although its only in 1 while true loop, there are multiple IF statements but these are acting as guards and are preventing the user from choosing something they are allergic to. However, they are still only part of a block of code doing 1 job, getting the starter selection choice from the user so these should all be in the same method.
- Methods being at the end of a program is the best professional practice because main is the entry point and when someone opens your file, they want to see what the program does, what ruins first and the core flow. Main being at the top makes this immediately visible, hence main being like a book index and the methods being the chapters they can read if they want more detail

- For void methods vs return methods. Void methods are used when you just want to print something and DON'T need to return a value. Return methods like int or String must return a value of that same type, so a double method must return a double value. These are used for your actual code
- You can make new variables within a method of a different type from the one your returning but you must still return the same variable type as what the method states, so you need to find a way to convert the variable to what you're returning within the method., for example if you have a Boolean method for a rerun loop of the whole of main (like in banking), you would declare Boolean true at the start of main and have main wrapped in a while (booleanName) so it keeps running while its true. Then at the end of main you would have a method like rerun and set booleanName = to rerun, so booleanName = rerun. So this method must return a boolean so in your method you'd ask the user to run again by choosing 1 to rerun or 2 to exit but this is an int so, you'd initialise int userChoice – scanner.nextInt() then use an IF statement like if user chooses 1, return true so you exit the method and the whole thing reruns because the condition hasn't changed, but if the user chooses 2, return false so then you exit the method, the boolean flag is now updated and the condition is met to exit the whole rerun loop.
- The main purpose of methods is to make them reusable and minimise the amount of code you write, as well as keeping main clean

General Notes

- Most of the code and notes was taken from the starters section, there were some things removed for mains and desserts, but these were all just copy and pasted from the starters part and most of the flow and decision making had already been done at this point
- The for loop that whole thing is wrapped in needs to be at the start of each section for mains and desserts because of the way I've written it, if I had included methods I could have just written it once and then called the methods for starters, mains and desserts below it. However, since the variable orders, used for how many orders the user wanted to place, was already declared at the beginning outside the loop, there was no need to ask this again since we have the value, so the for loop used could just be copy and pasted at the start of the mains and desserts section and it would be exactly the same as starters since this value was never changed after it was confirmed at the start, the for loop would run the same amount of times at each section, which as what we wanted.
- The allergy exclusion system was changed at the desserts part since dairy allergy covered all desserts and made them all inaccessible, so it didn't need to be 3 separate ones. It was changed to:

```
If (dairyAllergy && dessert != 4 && dessert != 5) {
    Print "Sorry, our set desserts have dairy so you cant order)
    Continue;
} //end if
```

- This just means if they have a dairy allergy and their dessert choice is not 4 or 5 (is not custom dessert or skip dessert), then you're not allowed to order this so skip the rest of the loop and try again
- You should never really use `scanner.nextBoolean();` for user centered console programs as it forces the user to enter exactly true or false and so makes your input very fragile since the user can only enter those 2 values. It also doesn't consume the newline properly which means it can cause problems, especially in larger programs. The best way to work around this is use `nextInt()` and convert it to a boolean, like I mentioned in the method best practice section. An example is shown below

```
private static boolean askLoyaltyCard(Scanner scanner) { 1usage
    while (true) { //Retry loop
        try {
            System.out.println("Do you have a loyalty card with us? \n1 = Yes \n2 = No"); //Ask user if they have a loyalt card
            int loyaltyChoice = scanner.nextInt();

            if (loyaltyChoice == 1) {
                System.out.println("Excellent, you'll have a 10% discount applied to your final price, thanks for sticking with us!");
                return true; //If user chooses 1, set loyaltyCard as true
            } else if (loyaltyChoice == 2) {
                System.out.println("No worries, you'll be charged full price for your order.");
                return false; //If user chooses 2, set loyaltyCard as false
            } else {
                System.out.println("Please choose either 1 or 2.");
            }
        } catch (Exception e) {
            System.out.println("Please choose 1 or 2.");
            scanner.next();
        }
    }
}
```

Here, the method is returning a boolean value but we use `.nextInt()` to accept the user input, either 1 or 2, and then use an IF statement to return a boolean based on the int the user entered, i.e., if user entered 1, return true

- The return keyword is used to exit a method and return a value to the method caller. It isn't always necessary in methods like void methods as these will just exit automatically after running since they don't need to return anything. The reason you often see break and return in the same method is because there may be multiple loops in a method, and you may not want to return a value immediately, you may want to do it later in the method as you want other things to run so you'd use break instead to break out of the first loop as break only breaks out of the current loop, but won't exit the method. TLDR: return; exits the entire method immediately and break only exits the loop.
- With methods and parameters – what you're passing into the method, when you call the method in main, you only need to pass the variable name you're passing into the method, i.e.,
`String displayName = getDisplayName(chosenTitle, firstName, surname)`
 We are passing these 3 variables into the method, so we pass the variable names in here by putting them in the brackets, and separating them with the comma

Then when you're actually defining the method, you need to type both the variable type first and then the variable name i.e.,

```
Private static String getDisplayName (String title, String firstName, String surname) {
```

As you can see, this has both the variable type and variable name.

- Remember, scanner is also a variable so this has to be passed into methods as well. Just (scanner) in main and then (Scanner scanner) in the methods.
- **Regex means Regular Expression** and is basically a pattern used to check if text matches certain rules. It is used in the name and title methods in here to check if the user text matches the rules set, for example `[a-zA-Z ]+`. a-z = lowercase letters, A-Z = uppercase letters, there is a space after "Z" which means spaces are allowed, and + = one or more characters, so not empty. So basically, this means, only letters and spaces allowed.

An example of using regex to validate text would be:

```
If (!name.matches("[a-zA-Z ]+"))
```

This means if the name DOES NOT match the allowed pattern, which is only letters and spaces allowed.

It is useful because it lets you describe the whole rule or set the pattern you want allowed in one pattern instead of loads of individual checks.

The important takeaway from this task is:

- Regex = text validation pattern
- `.matches()` checks the pattern
- Symbols like `[]`, `+`, `*` have meanings.
- `.equals()` and `.matches()` are different syntax used to check different things.
 - `.equals()` checks if 2 Strings are exactly the same:
Name.equals("Raymond") would allow "Raymond" but not "raymond", since there is no capital R. So `.equals()` compares the full text literally.
 - `.matches()` is used to check if the string follows a PATTERN (i.e., used alongside regex):
Name.matches("[a-zA-Z]+"). This is not checking for JUST ONE WORD, but its checking if the input matches a pattern, i.e., letters only, spaces and one or more characters.
The best way to think about them is `.equals()` checks an exact thing, and `.matches()` checks if it matches a rule or pattern, which is why it's used with regex rules. `.equals()` is better for checking fixed choices, like "yes" or "no", whereas `.matches()` is better for validating user input patterns, like their name can only contain letters and spaces and must not be empty, because you don't know their exact name, but you know the rules allowed for a name. So use `.equals()` for exact values and `.matches()` for regex patterns and rules.

Java switches

- Java has 2 different kinds of switch. Traditional switch statement and the switch expression.

1. Switch Statement:

This is the older style and is used to DO actions, print things or run code. An example would look like:

```
switch (choice) {  
case 1:  
System.out.println("Starter");  
break;
```

```
case 2:  
System.out.println("Main");  
break;
```

```
default:  
System.out.println("Invalid choice");  
}
```

This means if choice was 1, the program would print starter, and so on. A Traditional switch statements usually need break to prevent fallthrough into the next case.

A switch STATEMENT does NOT return a value, it just runs code and performs action.

2. Switch Expression

Newer Java feature which is used when you want the switch to RETURN a value. For example:

```
String title = switch (choice) {  
case 1 -> "Mr";  
case 2 -> "Mrs";  
default -> "";  
};
```

This DOES return a value

A switch expression also uses -> (arrow syntax) instead of : AND it can be assigned to variables

- The easiest way to compare them is that a switch statement is used to DO something like print x if user enters y, whereas a switch expression is used to calculate or return something, so it would best be used in methods which return something as well.
- The keyword yield is only used for switch expressions and is needed when you use a block ... {...} that must return a value. This usually happens when the case contains multiple lines of code.

```
- String result = switch (x) {  
case 1 -> "Hello";  
default -> "Bye";  
};
```

This DOES NOT need yield because it's one line with one value

```

- String result = switch (x) {
    case 1 -> {
        System.out.println("Hello");
        yield "Mr";
    }

    default -> {
        yield "";
    }
};

```

This DOES need yield because it contains multiple lines

- Java needs one final value returned from the block, this is what yield does.
- You use a block case with switch expressions when a switch expression case needs to do more than just return one simple value. For example, when the case needs:
 - Multiple lines
 - Variables
 - Printing something
 - Logic
 - Input
 - Calculations
 - Loops

So, the case at the top of page is yielding a value BUT is also printing something, hence why it needs a block because it isn't just case -> value. So, Java needs to know what value this whole block return should. Think of it like methods, where you need a return statement, so Java knows what value is being returned from the method, a switch expression needs yield to know what value is being returned from the block. So, if there are multiple actions/statements in a switch expression case, it needs to be in a block and because a switch expression must return a value, the block needs yield.
- The default keyword can also be used at the end of a switch statement or expression. Default basically acts as a guard or input validation method, think of it like an else statement at the end of an if/else if. It's basically just saying if none of these cases match, then do this instead, exactly the same as with else, saying if none of the conditions are met in the if statement, do this. Default is used after all the cases have made and goes at the end of a switch. Like an else block as well, default ISNT always required with a switch if the input has already been validated by guards you added earlier, so if it is guaranteed to be valid input in the switch, then you don't need default.
- However, in reality, a switch statement doesn't need default, but a switch expression usually ALWAYS DOES. This is because in a switch statement, default is optional because it does NOT need to return anything and Java just moves on. But, in a switch expression, it must ALWAYS produce a value, so Java needs to know what should this expression evaluate to, this is called exhaustiveness. So, the rule is a switch expression must always be exhaustive, which means it must return something in EVERY POSSIBLE CASE, i.e., it cannot compile unless ALL possible paths are covered. This means either using default OR every possible

value being explicitly covered, which is practically impossible with Strings. It doesn't matter if your guards make invalid input impossible, Java still wants proof so Java will only be satisfied with a default in a switch expression so **YOU DO NEED IT WITH SWITCH EXPRESSIONS** in most cases. You can even yield "", i.e., return an empty string if you know your guards make the default section impossible to reach, as Java will still be satisfied as **SOMETHING** is being returned.

Reusing methods

- Methods are supposed to be reusable, which means I could have used only my starter method for both mains and desserts, as they can behave differently depending on the parameters you pass in.
- It may seem like they would have to be different because they're asking the user different things, i.e., mains or starters or desserts, however you can actually just pass different text/data into the parameters., as the actual structure of the code remained almost identical each time, it was only the data they changed.
- The key idea with this is that the logic or process stayed the same throughout, only the data changed:
 - Logic:
 - Show menu
 - Validate input
 - Confirm choice
 - Store order
 - Data:
 - Garlic bread
 - Fish and chips
 - Cheesecake

So, a single method could handle all of this depending on the parameters you passed in, if the logic is the same, the data should usually become parameters which will change how the method behaves.
- Think of it like a vending machine where the machine logic stays the same, i.e., display items, take the money, validate the choice and dispense the item but the PRODUCTS change, for example coke instead of Pepsi, but you feed different data into the same system
- So here, we would just have to pass the relevant allergies in as parameters as well as the starter and main choices
- The only problem is that the logic in desserts changed slightly, so it's not just different data but also a slightly different structure as all of the desserts contained dairy and this allergy covered them all. However, just because most of the method is reusable, this DOES NOT mean every single line must be identical. This is where methods become a bit more nuanced.
- The starter/main logic was very direct, one allergy blocks one menu item, however with dessert, one allergy blocked multiple menu options, BUT, even when some logic differs, you can still reuse MOST of the method, which is common professional practice. A reusable method doesn't mean every line is identical, it means most of the process is shared.
- There are different ways you can design reusable systems/methods even with slightly different logic, such as allowing small custom checks, passing in the allergy rules here as parameters/data, which is closer to a professional design, or making each course its own object/class with its own validation rules, however I will learn more about this in V3.

General Notes 2

```
• case 4 -> {  
    System.out.println("Please enter the name of your custom starter:");  
    String customStarter = scanner.nextLine();  
    starterName = customStarter;  
    starterPrice = 11.50;  
}
```

With this part in the code, the yellow sections could just be omitted and replaced with `customStarter = scanner.nextLine();`

This is because if a variable is only:

- storing a value briefly
- immediately copied somewhere else
- never reused

Then it is often unnecessary

- The capitalise section was used to format user input, string manipulation and making the output cleaner and more professional.
- Capitalise method:

```
private static String capitalise(String input) {  
    if (input == null || input.isEmpty()) return input;  
    return text.substring(0,1).toUpperCase() +  
        text.substring(1).toLowerCase();  
}
```

This section then then plugged into:

```
- firstName = capitalise(firstName);
```

What this does is the user types rAYMOND, it changes it to Raymond. This is because `substring(0,1)` gets r then `toUpperCase()` makes R. then `substring(1)` makes AYMOND become aymond. The important part to remember here is `return firstPart + secondPart`, THESE ARE ALL ON ONE LINE AND YOU USE A +.

- The null and `.empty()` parts are important as well.
 - `== null` checks if the String object exists at all (doesn't really make sense for now but JUST REMEMBER IT HAS TO BE THERE), and `isEmpty()` checks if the string contains zero characters. Both are needed because Java will crash if `text.isEmpty()` doesn't have `text==null`, as to Java `isEmpty()` without `== null` is like asking it to check if there are 0 characters in nothing.
- The important part to remember even though it doesn't make sense now is `null = nothing stored` and `isEmpty()` checks for empty text stored, i.e., 0 characters.
- This would NOT be needed in my program however as the input could never be null since `capitalise` (the method above in blue) was being called within a different method after the input had already been entered, so even if it was empty, it would still exist. However, it is good practice to use as it becomes more relevant later on in things like databases, files, APIs, objects/classes and so on. At this stage it's just defensive programming and good practice, like a guard.

```

• private static String getDisplayName(String title, String firstName,
String surName) {
    String fullName = capitalise(firstName) + " " +
capitalise(surName);
    if (title == null || title.isEmpty()) {
        return fullName;
    }
    return title + " " + fullName;
}

```

- The above method actually calls another method within it, the capitalise one explained above. As you can see, it is not required to pass a method into another method as a parameter to use it within that method. A method can call another method within it.
- It just creates a new variable to return = to capitalise firstName and surname.
- It also returns only the fullname if the user doesn't enter a title
- @NotNull in IntelliJ next to your methods is just the IDE trying to add extra safety and documentation to the methods. It means this value should never be null. This is called an annotation which is extra instructions for IntelliJ, the compiler, libraries and other devs.
- It basically thought capitalise would crash if text == null because of text.substring() as the method expects a non-null string. It appeared in this program because my methods returned Strings, expected valid Strings and used String methods that can crash on null. It's mainly a warning to help detect mistakes and improve readability
- %s is used within system.out.printf and is a placeholder for a String as you put %s where your variable would be and then after the quotation marks, you put , variableName, variableTwoName. As you can see, the values go in order, for example:
System.out.printf("Dear %s, your total to pay with no discount is: £%.2f", displayName, finalPrice);
- printf is used instead of println to format things more cleanly, especially when there are multiple variables and you want to add signs like £.
- Other common placeholders include:
 - %s = String
 - %d = int
 - %f = double
 - %.2f = rounded to 2 d.p

```

• private static String askTitle(Scanner scanner) {
    String title;
    while (true) {
        System.out.println("And what is your title? Please choose
from Mr, Mrs, Miss, or Other.");
        title = scanner.nextLine().trim();

        if (title.isEmpty()) {
            System.out.println("Please choose Mr, Mrs, Miss, or
Other.");
            continue;
        }
        if (!title.matches("[a-zA-Z ]+")) {
            System.out.println("Please use letters and spaces
only.");
            continue;
        }
        if (!title.equals("Mr") && !title.equals("Mrs") &&
!title.equals("Miss") && !title.equals("Other")) {
            System.out.println("Please choose Mr, Mrs, Miss, or
Other.");
            continue;
        }
        break;
    }

    return switch (title) {
        case "Mr", "Mrs", "Miss" -> title;
        case "Other" -> {
            System.out.println("Please enter your preferred title, if
you don't want one, leave this blank.");
            String customTitle = scanner.nextLine().trim();
            yield customTitle;
        }
        default -> { //Switch expression must have a default option
even if it's unreachable with your guards
            System.out.println("Please choose one of the
aforementioned options.t");
            yield "";
        }
    };
}

```

- As you can see from this section in the if statements, there are some important guards that can be used to validate user input with Strings. The first two are both regex terms I explained earlier, .isEmpty checks if the user has inputted nothing and .matches checks if the input matches the rules we set out, i.e., it is only lowercase and uppercase characters entered, the space inside the [] checks spaces entered and + if it is more than 1 character. The last if statement checks if it doesn't match the options set out, then please try again and select one of these options. The break statements breaks out of the loop upon successful input. It is at the end here because you only want to break out upon successful input, after all of our guards have been checked.
- Another important thing here is the return statement, The switch expression itself evaluates to/returns a value, and the method returns the result of the switch expression, which is something you can do. We use the yield option to

return a different variable (yield will return it outside the switch but since we are returning the relevant case from the switch, this will return customTitle).

- You should always use `scanner.next()` when validating numerical input with `SCANNER` and try/catch statements after the printed message in the catch section. For example:

```
• } catch (Exception e) { //Catch invalid input
    System.out.println("Please enter a whole number between from 1 to 5.");
    scanner.next();
```

This just clears the bad token (previous invalid input) and allows a new one to be entered, or more specifically, stays in the scanner buffer. This should ALWAYS be used, especially with number variables like `int` and `double` (as this is where try/catch is mainly used), because the program can get stuck reading the first invalid input if it is not cleared and run in an infinite loop of constantly taking the invalid input as what's been entered if its not cleared.